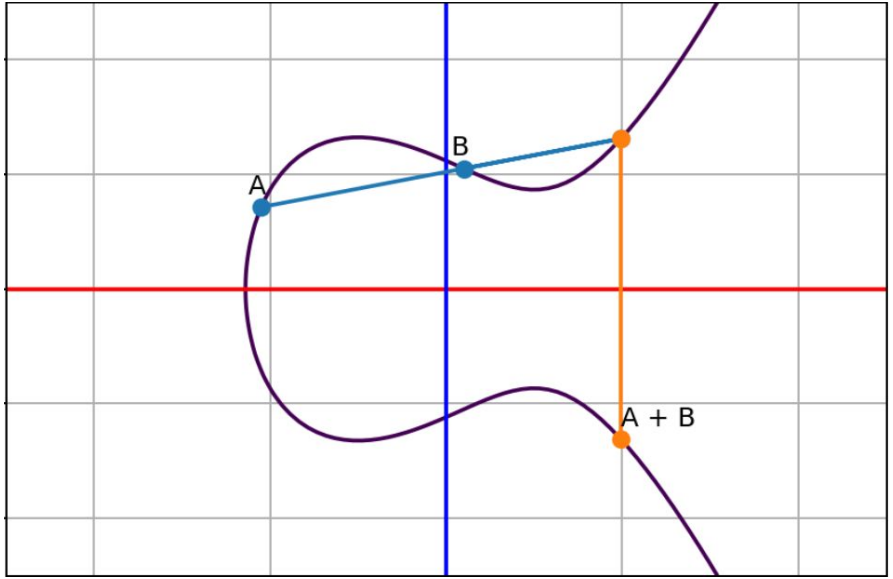
Performance Benchmark of Elliptic Curve Cryptography on Microblaze Processor

A modern public-key encryption technique

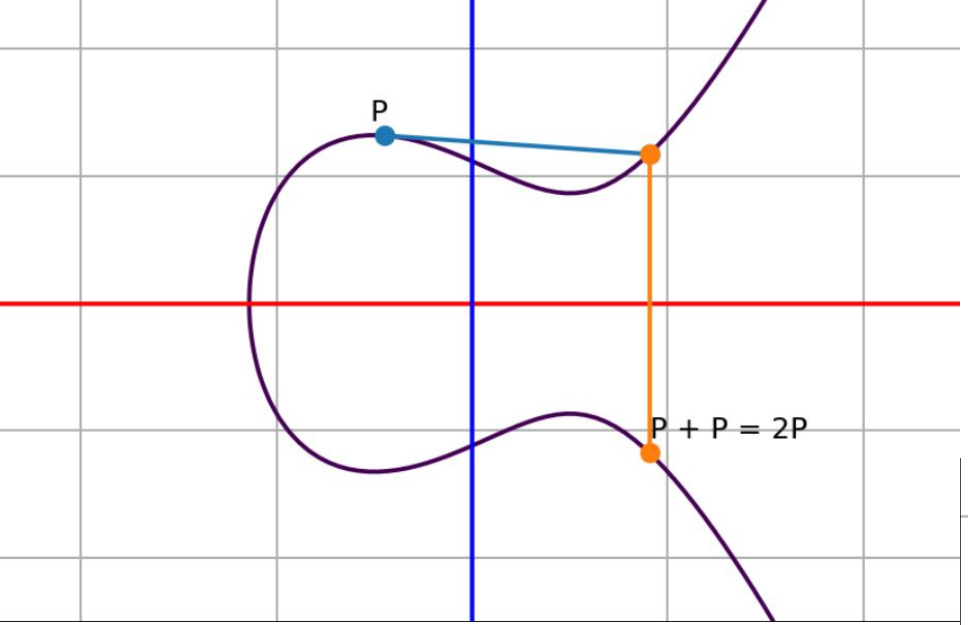
Tommy Duong, Sikan Fu, Torin Lee, Denny Nguyen

What is Elliptic Curve Cryptography?

- Uses math on curves to secure data
- Strong security with smaller keys than RSA
- Used in banking, HTTPS, messaging, IoT

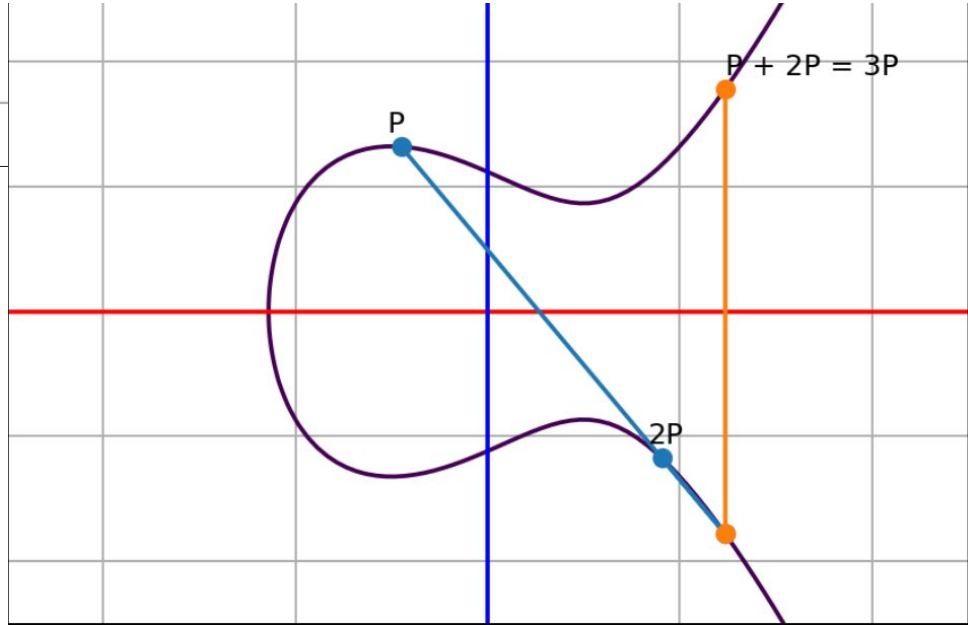


y^2=x^3+ax+b



1.) Pick a point P
2.) Create a tangent line (blue)
3.) Reflect that point across the x-axis

Now we can repeat the same step more, this time, we start from 2P, back to the original point P



Project Overview

Goal: To evaluate the efficiency of ECC (secp256r1, secp192r1 and secp160r1) on Microblaze processor using Nexys A7-100T to determine its viability.

Why: To simulate real embedded systems and determine whether strong encryption like ECC can be efficiently used on resource-constrained systems.

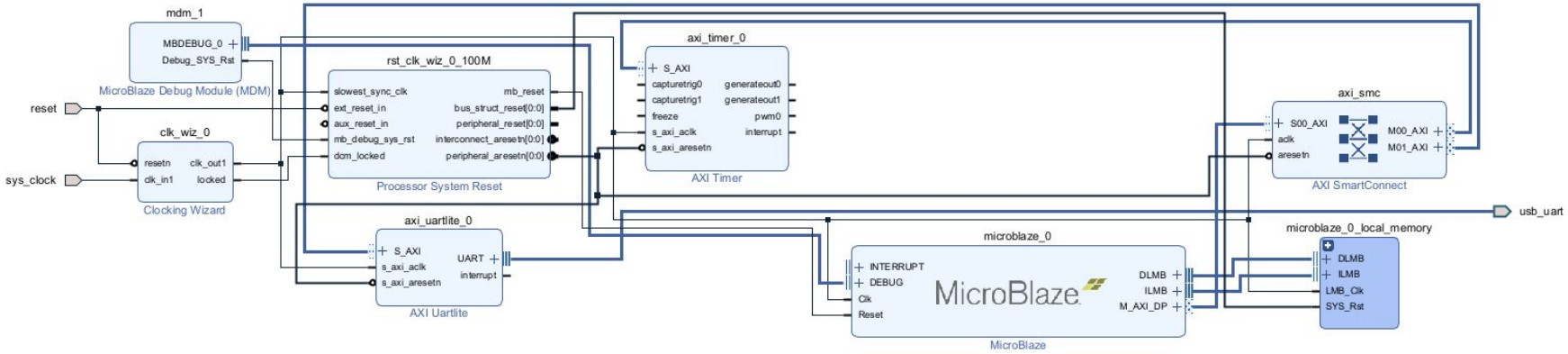
Benchmarks:

- **Latency and Throughput:** Cycles required for Key Generation, ECDSA Signing, and Verification.
- **Comparison:** A baseline test against the key sizes of secp256r1, secp160r1, and secp192r1 prove ECC's efficiency in resource-constrained environments.

Microblaze processor block diagram

Vivado microblaze processor for .xsa hardware file

- Clocking module
- AXI timer
- UART



Benchmark for secp160r1, secp192r1, and secp256r1

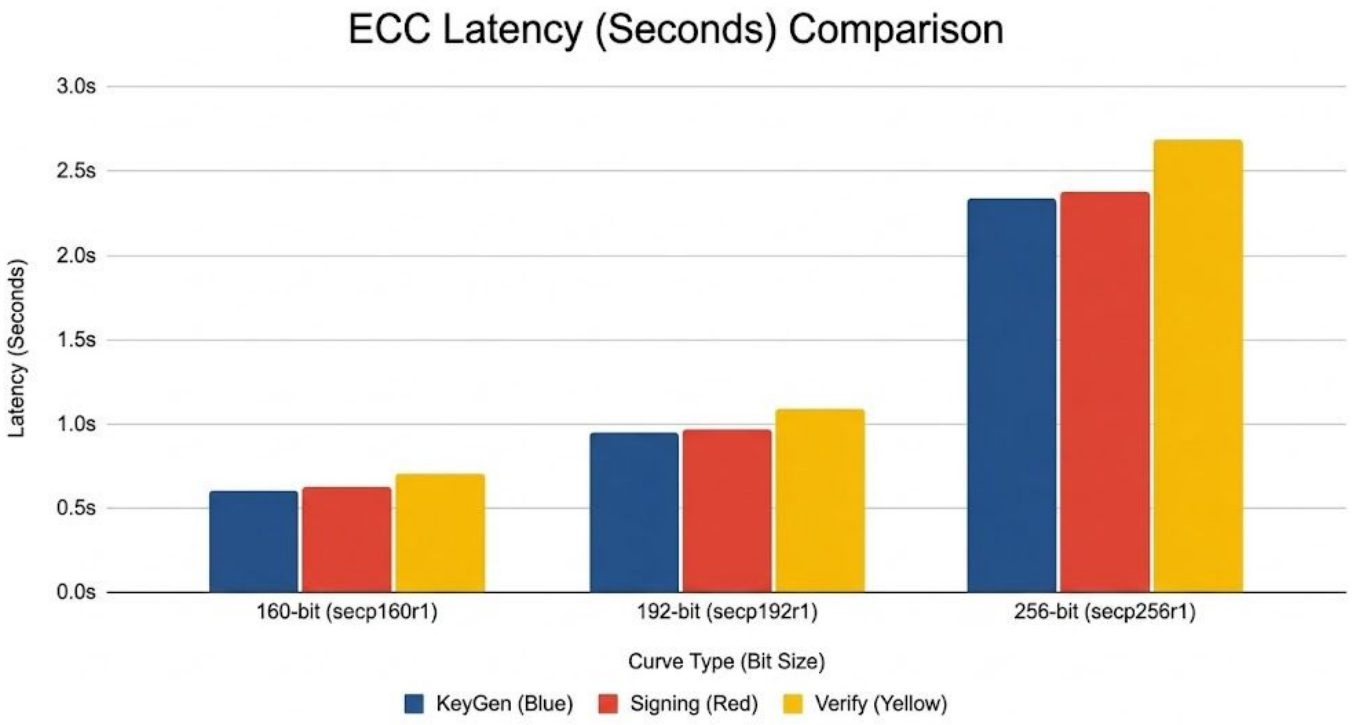
```c
void run_benchmark(const struct uECC_Curve_t * curve, char* name) {
    uint8_t public[64], private[32], hash[32] = {1}, sig[128];
    u32 start, end;
    int status;

    xil_printf("\r\n--- Benchmarking %s ---\r\n", name);

    // 1. Key Generation
    XTmrCtr_Start(&Timer, 0);
    start = XTmrCtr_GetValue(&Timer, 0);
    status = uECC_make_key(public, private, curve);
    end = XTmrCtr_GetValue(&Timer, 0);
    xil_printf("  KeyGen: %lu cycles (Status: %d)\r\n", end - start, status);

    // 2. Signing
    start = XTmrCtr_GetValue(&Timer, 0);
    status = uECC_sign(private, hash, 32, sig, curve);
    end = XTmrCtr_GetValue(&Timer, 0);
    xil_printf("  Signing: %lu cycles (Status: %d)\r\n", end - start, status);

    // 3. Verification
    start = XTmrCtr_GetValue(&Timer, 0);
    status = uECC_verify(public, hash, 32, sig, curve);
    end = XTmrCtr_GetValue(&Timer, 0);
    xil_printf("  Verify: %lu cycles (Status: %d)\r\n", end - start, status);
}
```

Nexys A7 Output Cycles

Bit Size	Curve Name	Operation	Cycles	Time (Seconds)
160-bit	**secp160r1**	KeyGen	60,126,979	0.60s
		Signing	62,511,860	0.62s
		Verify	70,205,010	0.70s
192-bit	**secp192r1**	KeyGen	94,129,079	0.94s
		Signing	96,369,492	0.96s
		Verify	108,902,654	1.09s
256-bit	**secp256r1**	KeyGen	234,176,849	2.34s
		Signing	238,078,119	2.38s
		Verify	267,640,482	2.68s

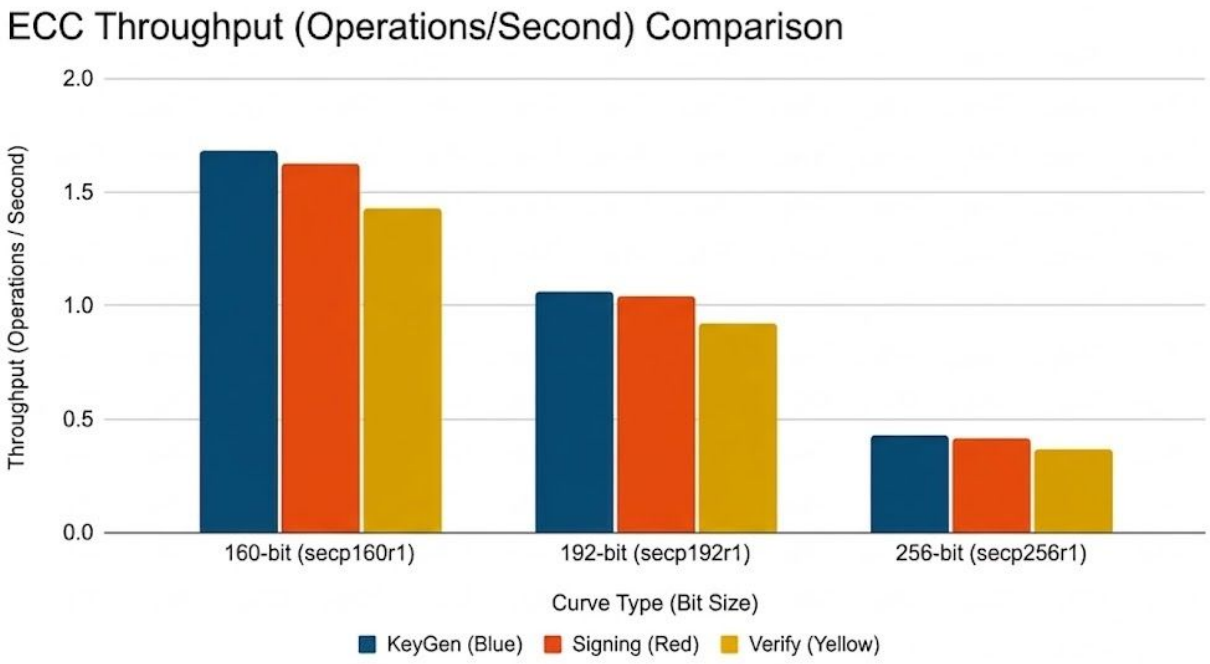
```
OUTPUT ×   COM5 (FTDI) ×   TASK: XSDB CONSOLE   C:\WINDOWS\SYSTEM32\CMD.EXE ×
Opened with baud rate: 9600

=== Nexys A7 ECC Full Suite Benchmark ===

--- Benchmarking 160-bit ---
  KeyGen: 60126979 cycles (Status: 1)
  Signing: 62511860 cycles (Status: 1)
  Verify: 70205010 cycles (Status: 1)

--- Benchmarking 192-bit ---
  KeyGen: 94129079 cycles (Status: 1)
  Signing: 96369492 cycles (Status: 1)
  Verify: 108902654 cycles (Status: 1)

--- Benchmarking 256-bit ---
  KeyGen: 234176849 cycles (Status: 1)
  Signing: 238078119 cycles (Status: 1)
  Verify: 267640482 cycles (Status: 1)

Done!
```

Latency


ECC Latency (Seconds) Comparison
3.0s
2.5s
2.0s
1.5s
1.0s
0.5s
0.0s
Latency (Seconds)
160-bit (secp160r1)
192-bit (secp192r1)
256-bit (secp256r1)
Curve Type (Bit Size)
KeyGen (Blue)
Signing (Red)
Verify (Yellow)

Throughput

- Key gen- 1 operation
- Signing- 4 operations
- Verifying- 5 operations



Conclusion

Key takeaways:

- ECC is viable on the Microblaze
- Performance increases significantly with key size
- Verification is the most computationally expensive operation
- Strong tradeoff between security and performance

Thank You!

Questions?